

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

OPERATING SYSTEMS

Submitted by

R K MADAN (1WA24CS225)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by R K MADAN (1WA24CS225), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Non Preemptive) c) SRTF (SJF Preemptive) d) Priority (Non Preemptive) e) Priority (Preemptive) f) Round Robin	01-41
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue	42-50
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	51-71
4.	Write a C program to simulate producer-consumer problem using semaphores	72-84
5.	Write a C program to simulate the concept of Dining Philosophers problem.	85-106
6.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	107-117
7.	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	118-135
8.	Observation Book Index Page	136

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

LAB PROGRAM – 1:

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

- a) FCFS
- b) SJF (Non Preemptive)
- c) SRTF (SJF Preemptive)
- d) Priority (Non Preemptive)
- e) Priority (Preemptive)
- f) Round Robin

a) FCFS:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i;
    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];
    float avg_tat = 0, avg_wt = 0, avg_rt = 0;

    printf("Enter Arrival Time and Burst Time:\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
}
```

```

ct[0] = at[0] + bt[0];

for(i = 1; i < n; i++)
{
    if(ct[i-1] < at[i])
        ct[i] = at[i] + bt[i];
    else
        ct[i] = ct[i-1] + bt[i];
}

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = wt[i]; // FCFS: RT = WT

    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += rt[i];
}

avg_tat /= n;
avg_wt /= n;
avg_rt /= n;

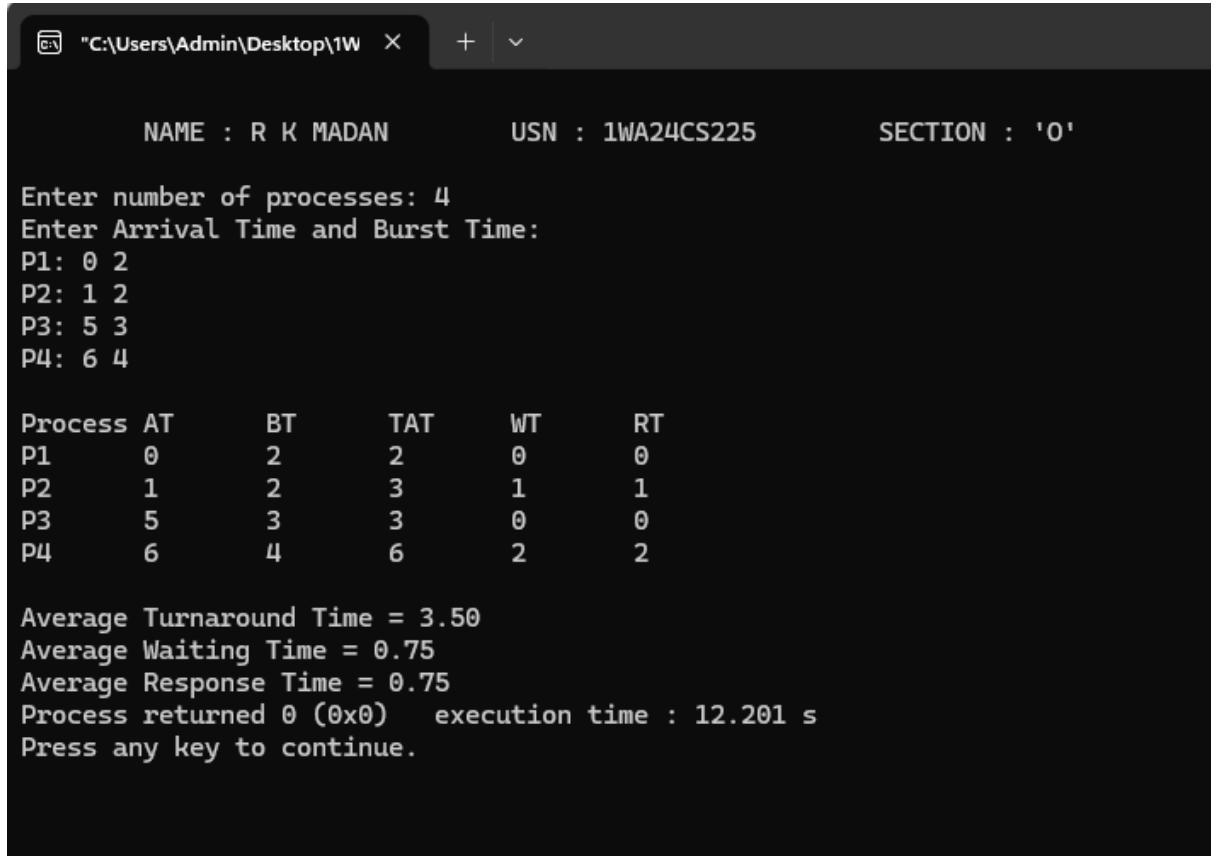
printf("\nProcess\tAT\tBT\tTAT\tWT\tRT\n");
for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], tat[i], wt[i], rt[i]);
}

printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage Waiting Time = %.2f", avg_wt);
printf("\nAverage Response Time = %.2f", avg_rt);

```

```
return 0;
}
```

OUTPUT:



```
"C:\Users\Admin\Desktop\1W" X + v

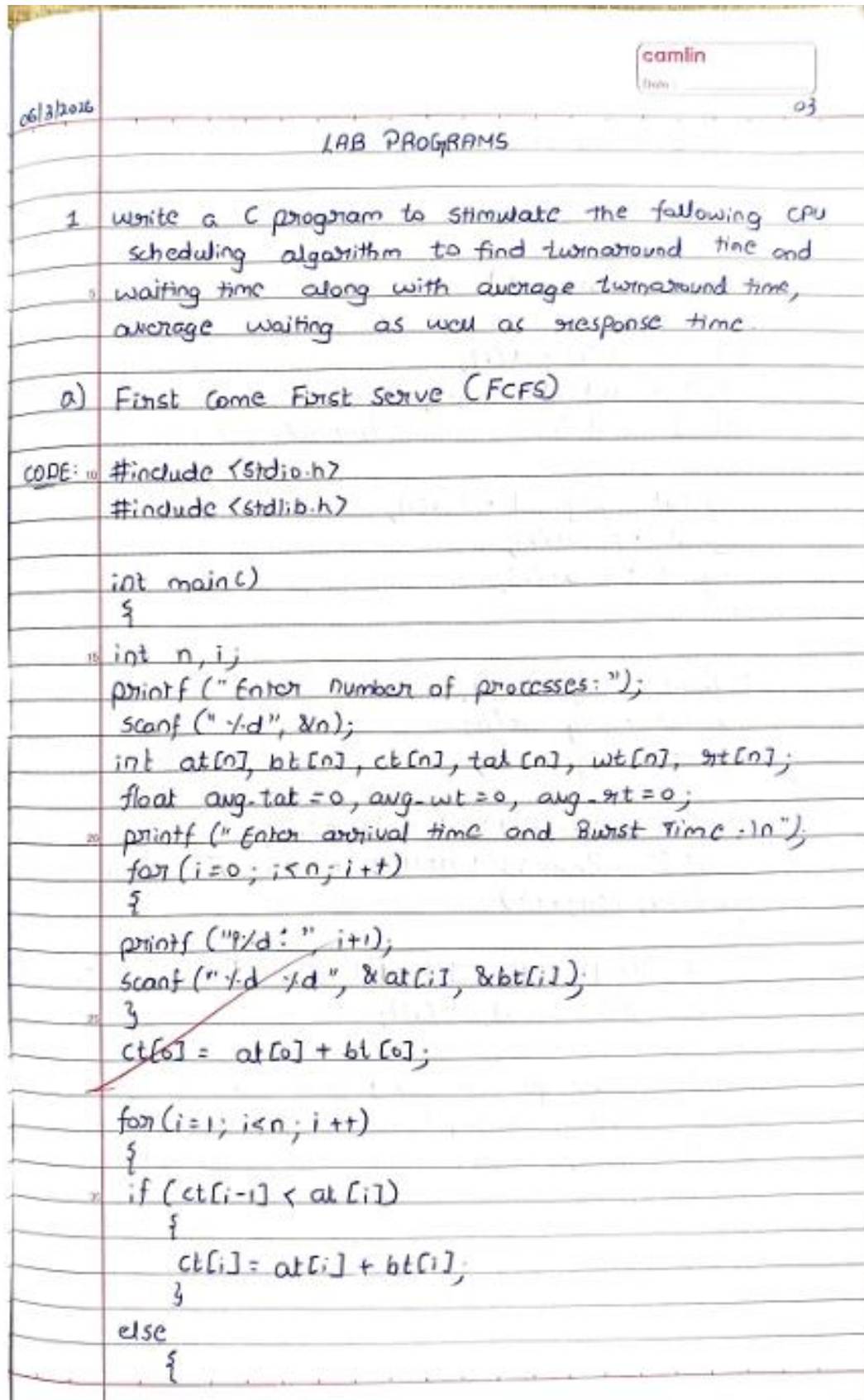
NAME : R K MADAN      USN : 1WA24CS225      SECTION : 'O'

Enter number of processes: 4
Enter Arrival Time and Burst Time:
P1: 0 2
P2: 1 2
P3: 5 3
P4: 6 4

Process AT      BT      TAT      WT      RT
P1      0      2      2      0      0
P2      1      2      3      1      1
P3      5      3      3      0      0
P4      6      4      6      2      2

Average Turnaround Time = 3.50
Average Waiting Time = 0.75
Average Response Time = 0.75
Process returned 0 (0x0)   execution time : 12.201 s
Press any key to continue.
```

OBSERVATION COPY:



```

    ct[i] = ct[i-1] + bt[i];
}
}

for (i=0; i<n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    rt[i] = wt[i]; // for fcts wt = rt
}

avg_tat = avg_tat + tat[i];
avg_wt += wt[i];
avg_rt += rt[i];
}

avg_tat = avg_tat / n;
avg_wt = avg_wt / n;
avg_rt = avg_rt / n;

printf("In Process |t| AT |t| BT |t| TAT |t| WT |t| RT |n");
printf("In Process |t| AT |t| BT |t| TAT |t| WT |t| RT |n");
for (i=0; i<n; i++)
{
    printf("P%d |t| %d |t| %d |t| %d |t| %d |t| %d |n", i+1, at[i],
    bt[i], tat[i], wt[i], rt[i]);
}

printf("In Average Turnaround Time = %.2f", avg_tat);
printf("In Average Waiting Time = %.2f", avg_wt);
printf("In Average Response Time = %.2f", avg_rt);
return 0;
}

```


Output:

Enter number of processes: 4

Enter arrival Time and Burst Time:

P₁ : 0 2

P₂ : 1 2

P₃ : 5 3

P₄ : 6 4

Process	AT	BT	TAT	WT	RT
P ₁	0	2	2	0	0
P ₂	1	2	3	1	1
P ₃	5	3	3	0	0
P ₄	6	4	6	2	2

Average Turnaround Time = 3.50

Average Waiting Time = 0.75

Average Response Time = 0.75

✓
6/5/4

b) SJF (Non Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
    int n,i,j;
    int pid[10],at[10],bt[10],ct[10],tat[10],wt[10];
    int finished[10]={0};
    int current_time=0,completed=0,min;
    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }

    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
        scanf("%d",&bt[i]);

    //Sorting according to arrival time
    for(i=0;i<n-1;i++)
    {
```

```

for(j=i+1;j<n;j++)
{
    if(at[i]>at[j])
    {
        int temp;

        temp=at[i];
        at[i]=at[j];
        at[j]=temp;

        temp=bt[i];
        bt[i]=bt[j];
        bt[j]=temp;

        temp=pid[i];
        pid[i]=pid[j];
        pid[j]=temp;
    }
}

```

//Scheduling

```

while(completed<n)
{
    min=-1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && finished[i]==0)
        {
            if(min==-1 || bt[i]<bt[min])

```

```

        min=i;
    }
}

if(min==-1)
{
    current_time++;
}
else
{
    ct[min]=current_time+bt[min];
    tat[min]=ct[min]-at[min];
    wt[min]=tat[min]-bt[min];

    current_time=ct[min];

    finished[min]=1;
    completed++;

    avg_tat+=tat[min];
    avg_wt+=wt[min];
}
}

printf("\nSJF Non-Preemptive Scheduling\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

```

```

    }

    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

    return 0;
}

```

OUTPUT:

```

NAME : R K MADAN      USN : 1WA24CS225      SECTION : 'O'

Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 3 4 6

SJF Non-Preemptive Scheduling
PID    AT    BT    CT    TAT    WT
P1     0     7     7     7     0
P3     3     4    11     8     4
P4     5     6    20    15     9
P2     8     3    14     6     3

Average Turnaround Time = 9.00
Average Waiting Time = 4.00

Process returned 0 (0x0)   execution time : 25.889 s
Press any key to continue.

```

OBSERVATION COPY:

6/3/2026

camlin

Date:

1. b

Write a C program to simulate the following CPU scheduling algorithm to find TAT, WT, along with average TAT & average WT

b) Shortest Job First (SJF) [Non-Preemptive]

```
CODE:- #include <stdio.h>
int main ()
{
    int n, i, j;
    int pid[10], at[10], bt[10], ct[10], tat[10], wt[10];
    int finished[10] = {0};
    int current_time = 0, completed = 0, min;
    float avg_tat = 0, avg_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time: \n");
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Time: \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
    }

    // sorting
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (at[i] > at[j])
            {
                int temp;
                temp = at[i];
            }
        }
    }
}
```

```
at[i] = at[j];  
at[j] = temp;
```

```
temp = bt[i];  
bt[i] = bt[j];  
bt[j] = temp;
```

```
temp = pid[i];  
pid[i] = pid[j];  
pid[j] = temp;
```

```
}
```

```
}
```

```
}
```

```
// Scheduling
```

```
while (completed < n)  
{
```

```
    min = -1;
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (at[i] <= current_time && finished[i] == 0)
```

```
        {
```

```
            if (min == -1 || bt[i] < bt[min])
```

```
                min = i;
```

```
        }
```

```
    }
```

```
    if (min == -1)
```

```
    {
```

```
        current_time++;
```

```
    }
```

```
    else
```

```
    {
```

```
        ct[min] = current_time + bt[min];
```

```
        tat[min] = ct[min] - at[min];
```

```
        wt[min] = tat[min] - bt[min];
```

```

current_time = ct[min];
finished[min] = 1;
completed++;
avg_tat += tat[min];
avg_wt += wt[min];
}
}
printf("\n SJF Non-Preemptive Scheduling\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0; i<n; i++)
{
printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i],
      bt[i], ct[i], tat[i], wt[i]);
}
printf("\n Average Turnaround Time = %.2f", avg_tat/n);
printf("\n Average Waiting Time = %.2f", avg_wt/n);
return 0;
}

```

OUTPUT:

Enter number of processes: 4

Enter Arrival Time:

0 8 3 5

Enter Burst Time:

7 3 4 6

SJF Non-Preemptive Scheduling

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P3	3	4	11	8	4
P4	5	6	20	15	9
P2	8	3	14	6	3
P4					

Average Turnaround Time = 9.00

Average waiting Time = 4.00

c) SRTF (SJF Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
    int n,i,time=0,completed=0;
    int pid[10],at[10],bt[10],rt[10];
    int ct[10],tat[10],wt[10];
    int idx,min_rt;
    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }

    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&bt[i]);
        rt[i]=bt[i];
    }
}
```

```

}

while(completed<n)
{
    idx=-1;
    min_rt=9999;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && rt[i]>0 && rt[i]<min_rt)
        {
            min_rt=rt[i];
            idx=i;
        }
    }

    if(idx==-1)
    {
        time++;
    }
    else
    {
        rt[idx]--;
        time++;

        if(rt[idx]==0)
        {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];

```

```

        avg_tat+=tat[idx];
        avg_wt+=wt[idx];
        completed++;
    }
}
}

printf("\nSJF Preemptive Scheduling (SRTF)\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}

printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

return 0;
}

```

OUTPUT:

```
"D:\OS Lab Programs\LAB PRI" X + v
NAME : R K MADAN      USN : 1WA24CS225      SECTION : '0'
Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 3 2 6

SJF Preemptive Scheduling (SRTF)
PID    AT    BT    CT    TAT    WT
P1      0     7     9     9     2
P2      8     3    12     4     1
P3      3     2     5     2     0
P4      5     6    18    13     7

Average Turnaround Time = 7.00
Average Waiting Time = 2.50

Process returned 0 (0x0)  execution time : 55.467 s
Press any key to continue.
|
```

OBSERVATION COPY:

camlin
Date: _____

05/03/2020 09

1. C Write a C program to simulate the following CPU scheduling algorithm to find TAT, WT along with average TAT & average WT

c) SRTF (Shortest Remaining Time First) / SJF Preemptive scheduling

CODE:-

```
#include <stdio.h>
int main ()
{
    int n, i, time = 0, completed = 0;
    int pid[10], at[10], bt[10], rt[10];
    int ct[10], tat[10], wt[10];
    int idx, min_rt;
    float avg_tat = 0, avg_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time: \n");
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Time: \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while (completed < n)
    {
        idx = -1;
        min_rt = 9999;
```

```

for (i=0; i<n; i++)
{
    if (at[i] <= time && rt[i] >= 0 && rt[i] < min_rt)
    {
        min_rt = rt[i];
        idx = i;
    }
}
if (idx == -1)
{
    time++;
}
else
{
    rt[idx]--;
    time++;
    if (rt[idx] == 0)
    {
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        avg_tat += tat[idx];
        avg_wt += wt[idx];
        completed++;
    }
}
printf("\n SJF Preemptive Scheduling (SRJF)\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (i=0; i<n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i],
        bt[i], ct[i], tat[i], wt[i]);
}
printf("\n Average Turnaround Time = %.2f", avg_tat/n);
printf("\n Average waiting Time = %.2f", avg_wt/n);
return 0;

```

3

Output:

Enter number of processes : 4

Enter Arrival Time :

0 8 3 5

Enter Burst Time :

7 3 2 6

SJF Preemptive Scheduling (SRTF)

PID	AT	BT	CT	TAT	WT
P1	0	7	9	9	2
P2	8	3	12	4	1
P3	3	2	5	2	0
P4	5	6	18	13	7

Average Turnaround Time = 7.00

Average Waiting Time = 2.50

d) Priority (Non Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
    int n,i;
    int pid[10],at[10],bt[10],pr[10];
    int ct[10],tat[10],wt[10];
    int finished[10]={0};
    int current_time=0,completed=0,max;
    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }

    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
        scanf("%d",&bt[i]);
```



```

printf("Enter Priority:\n");
for(i=0;i<n;i++)
    scanf("%d",&pr[i]);

while(completed<n)
{
    max=-1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && finished[i]==0)
        {
            if(max==-1 || pr[i]>pr[max])
                max=i;
        }
    }

    if(max!=-1)
        current_time++;
    else
    {
        ct[max]=current_time+bt[max];
        tat[max]=ct[max]-at[max];
        wt[max]=tat[max]-bt[max];

        current_time=ct[max];

        finished[max]=1;
        completed++;
    }
}

```

```

        avg_tat+=tat[max];
        avg_wt+=wt[max];
    }
}

printf("\nPriority Scheduling (Higher number = Higher priority)\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
}

printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

return 0;
}

```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W" X + v
NAME : R K MADAN      USN : 1WA24CS225      SECTION : 'O'

Enter number of processes: 5
Enter Arrival Time:
0 2 3 4 6
Enter Burst Time:
3 2 5 4 1
Enter Priority:
5 3 2 4 1

Priority Scheduling (Higher number = Higher priority)
PID   AT   BT   PR   CT   TAT  WT
P1    0    3    5    3    3    0
P2    2    2    3    5    3    1
P3    3    5    2   14   11    6
P4    4    4    4    9    5    1
P5    6    1    1   15    9    8

Average Turnaround Time = 6.20
Average Waiting Time = 3.20

Process returned 0 (0x0)   execution time : 94.714 s
Press any key to continue.
|
```

OBSERVATION COPY:

13/03/2016

1.d write a C Program to simulate the following CPU scheduling algorithm to find TAT, WT, along with average TAT & average WT.

d) Priority scheduling (Higher Number = Higher Priority)
(Non-Preemptive)

CODE:- #include <stdio.h>
int main()
{
10 int n, i;
int pid[10], at[10], bt[10], pr[10];
int ct[10], tat[10], wt[10];
int finished[10] = {0};
int current_time = 0, completed = 0, max;
15 float avg_tat = 0, avg_wt = 0;
printf("Enter number of processes : ");
scanf("%d", &n);
printf("Enter Arrival Time : \n");
for (i = 0; i < n; i++)
20 {
pid[i] = i + 1;
scanf("%d", &at[i]);
}
printf("Enter Burst Time : \n");
25 for (i = 0; i < n; i++)
{
scanf("%d", &bt[i]);
}
for (i = 0; i < n; i++)
30 printf("Enter priority : \n");
for (i = 0; i < n; i++)
{
scanf("%d", &pr[i]);
}
}

```

while (completed < n)
{
    max = -1; // for lower number = highest priority
    min = -1;
    for (i = 0; i < n; i++)
    {
        if (at[i] <= current_time && finished[i] == 0)
        {
            if (max == -1 || pr[i] > pr[max])
            /* for lower number = higher priority
            if (min == -1 || pr[i] < pr[min])
                min = i;
            */
        }
        max = i;
    }
    if (max == -1) // if (min == -1)
    {
        current_time++;
    }
    else
    {
        ct[max] = current_time + bt[max];
        tat[max] = ct[max] - at[max];
        wt[max] = tat[max] - bt[max];
        current_time = ct[max];
        finished[max] = 1;
        completed++;
        avg_tat += tat[max];
        avg_wt += wt[max];
    }
}

```

```

printf("\n Priority scheduling (Higher number = Higher
priority)\n");
printf(" P ID | A T | B T | P R | C T | T A T | W T | \n");
for (i=0; i<n; i++)
{
printf(" P %d | A %d | B %d | P %d | C %d | T %d | W %d | \n",
pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
printf("Average Turnaround Time = %.2f", avg_tat/n);
printf("Average Waiting Time = %.2f", avg_wt/n);
return 0;
}

```

Output:-

Enter number of processes : 5

Enter Arrival Time :

0 1 2 3 4

Enter Burst Time :

4 3 1 5 2

Enter Priority :

2 3 4 5 5

Priority Scheduling (Higher number = Higher priority)

PID	AT	BT	PR	CT	TAT	WT
P1	0	4	2	4	4	0
P2	1	3	3	15	14	11
P3	2	1	4	12	10	9
P4	3	5	5	9	6	1
P5	4	2	5	11	7	5

Average Turnaround Time = 8.20

Average Waiting Time = 5.20

e) Priority (Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
    int n,i;
    int pid[10],at[10],bt[10],pr[10],rt[10];
    int ct[10],tat[10],wt[10];
    int completed=0,current_time=0,min;
    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }

    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&bt[i]);
        rt[i]=bt[i];
    }
}
```

```

}

printf("Enter Priority:\n");
for(i=0;i<n;i++)
    scanf("%d",&pr[i]);

while(completed<n)
{
    min=-1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && rt[i]>0)
        {
            if(min==-1 || pr[i] < pr[min])
                min=i;
        }
    }

    if(min==-1)
        current_time++;
    else
    {
        rt[min]--;
        current_time++;

        if(rt[min]==0)
        {
            ct[min]=current_time;
            tat[min]=ct[min]-at[min];

```



```

        wt[min]=tat[min]-bt[min];

        completed++;

        avg_tat+=tat[min];
        avg_wt+=wt[min];
    }
}

printf("\nPriority Scheduling (Preemptive) Higher the Number, Higher the Priority\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);

printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

return 0;
}

```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W X + v

NAME : R K MADAN      USN : 1WA24CS225      SECTION : 'O'

Enter number of processes: 7
Enter Arrival Time:
0 1 3 4 5 6 10
Enter Burst Time:
8 2 4 1 6 5 1
Enter Priority:
3 4 4 5 2 6 1

Priority Scheduling (Preemptive) Higher the Number, Higher the Priority
PID   AT   BT   PR   CT   TAT  WT
P1    0    8    3    15   15   7
P2    1    2    4    17   16   14
P3    3    4    4    21   18   14
P4    4    1    5    22   18   17
P5    5    6    2    12    7    1
P6    6    5    6    27   21   16
P7   10    1    1    11    1    0

Average Turnaround Time = 13.71
Average Waiting Time = 9.86

Process returned 0 (0x0)   execution time : 54.150 s
Press any key to continue.
```

OBSERVATION COPY:

13/03/2026

camlin
Date: _____

1.e write a C Program to simulate the following CPU scheduling algorithm to find TAT, WT along with average TAT and average WT

e) Priority Scheduling (Preemptive)
Note (Higher Number = Higher priority)

CODE:-

```
#include <stdio.h>
int main()
{
    int n, i;
    int pid[10], at[10], bt[10], pr[10], rt[10], ct[10], tat[10];
    int wt[10], completed = 0, current_time = 0, max;
    float avg_tat = 0, avg_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival time: \n");
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Time: \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter Priority: \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &pr[i]);
    }
```

```

while (completed < n)
{
    max = -1; // for lower no = higher priority min = -1
    for (i = 0; i < n; i++)
    {
        if (at[i] <= current_time && pt[i] > 0)
        {
            if (max == -1 || pt[i] > pt[max])
                max = i;
        }
        /* for lower number = higher priority
        if (at[i] <= current_time && pt[i] > 0)
        {
            if (min == -1 || pt[i] < pt[min])
                min = i;
        }
        */
        if (max == -1)
        {
            current_time++;
        }
        else
        {
            pt[max]--;
            current_time++;
            if (pt[max] == 0)
            {
                ct[max] = current_time;
                tat[max] = ct[max] - at[max];
                wt[max] = tat[max] - pt[max];
                completed++;
                avg_tat += tat[max];
                avg_wt += wt[max];
            }
        }
    }
}

```

```

printf("In Priority scheduling (Preemptive) \n");
printf("PID | AT | BT | PR | CT | TAT | WT | \n");
for (i=0; i<n; i++)
{
    printf("P%d | %d | %d | %d | %d | %d | %d | \n",
           pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}
printf("In Average Turnaround Time = %.2f", avg_tat/n);
printf("In Average waiting Time = %.2f \n", avg_wt/n);
return 0;
}

```

output:

Enter number of processes: 7

Enter Arrival Time:

0 1 3 4 5 6 10

Enter Burst Time:

8 2 4 1 6 5 1

Enter Priority:

3 4 4 5 2 6 1

Priority scheduling (Preemptive) Higher the Number, Higher the Priority

PID	AT	BT	PR	CT	TAT	WT
P1	0	8	3	15	15	7
P2	1	2	4	17	16	14
P3	3	4	4	21	18	14
P4	4	1	5	22	18	17
P5	5	6	2	12	7	1
P6	6	5	6	27	21	16
P7	10	1	1	11	1	0

Average Turnaround Time = 13.71

Average waiting Time = 9.86

f) Round Robin

CODE:

```
#include <stdio.h>

int main()
{
    int n,i;
    int pid[10],at[10],bt[10],rt[10];
    int ct[10],tat[10],wt[10];
    int tq,current_time=0,completed=0;
    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        scanf("%d",&at[i]);
    }

    printf("Enter Burst Time:\n");
    for(i=0;i<n;i++)
    {
```

```

scanf("%d",&bt[i]);

rt[i]=bt[i];
}

printf("Enter Time Quantum: ");
scanf("%d",&tq);

int done;
while(completed<n)
{
    done=1;

    for(i=0;i<n;i++)
    {
        if(at[i]<=current_time && rt[i]>0)
        {
            done=0;

            if(rt[i] > tq)
            {
                current_time += tq;
                rt[i] -= tq;
            }
            else
            {
                current_time += rt[i];
                rt[i]=0;
            }
        }
    }
}

```

```

        ct[i]=current_time;
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];

        completed++;

        avg_tat+=tat[i];
        avg_wt+=wt[i];
    }
}

if(done==1)
{
    current_time++;
}

}

printf("\nRound Robin Scheduling\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}

```



```

printf("\nAverage Turnaround Time = %.2f",avg_tat/n);

printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

return 0;

}

```

OUTPUT:

```

NAME : R K MADAN      USN : 1WA24CS225      SECTION : '0'

Enter number of processes: 5
Enter Arrival Time:
0 1 2 3 4
Enter Burst Time:
5 3 1 2 3
Enter Time Quantum: 2

Round Robin Scheduling
PID   AT   BT   CT   TAT  WT
P1    0    5   14   14    9
P2    1    3   12   11    8
P3    2    1    5    3    2
P4    3    2    7    4    2
P5    4    3   13    9    6

Average Turnaround Time = 8.20
Average Waiting Time = 5.40

Process returned 0 (0x0)   execution time : 15.305 s
Press any key to continue.

```

OBSERVATION COPY:

camlin
Date: _____

19

13/03/2026

1.f. Write a C Program to simulate the following CPU scheduling algorithm to find TAT, WT along with average TAT and average WT

f). Round Robin scheduling

code:-

```
#include <stdio.h>
int main()
{
    int n, i;
    int pid[10], at[10], bt[10], rt[10], ct[10], tat[10], wt[10];
    int tq, current_time = 0, completed = 0;
    float avg_tat = 0, avg_wt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Arrival Time: \n");
    for (i = 0; i < n; i++)
    {
        pid[i] = i + 1;
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Time: \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
    int done;
    while (completed < n)
    {
        done = 1;
        for (i = 0; i < n; i++)
        {
```

```

if (at[i] <= current_time && gt[i] > 0)
{
    done = 0;
    if (gt[i] > tq)
    {
        current_time += tq;
        gt[i] = tq;
    }
    else
    {
        current_time += gt[i];
        gt[i] = 0;
        ct[i] = current_time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        completed++;
        avg_tat += tat[i];
        avg_wt += wt[i];
    }
}

if (done == 1)
{
    current_time++;
}

printf("In Round Robin Scheduling\n");
printf("Print AT | BT | CT | TAT | WT\n");
for (i = 0; i < n; i++)
{
    printf("P%d | t%d | t%d | t%d | t%d | t%d | t%d | t%d | n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i],
        avg_tat/n, avg_wt/n);
}

printf("In Average Turnaround Time = %.2f", avg_tat/n);
printf("In Average Waiting Time = %.2f", avg_wt/n);

```

return 0;
3

Output:

Enter number of processes: 5

Enter Arrival Time:

0 1 2 3 4

Enter Burst Time:

5 3 1 2 3

Enter Time Quantum: 2

Round 1 Time Quantum: 2

Round Robin scheduling

PID	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	12	11	8
P3	2	1	5	3	2
P4	3	2	7	4	2
P5	4	3	13	9	6

Average Turnaround Time = 8.20

Average Waiting Time = 5.40

✓ 6/15

LAB PROGRAM - 2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>

int main()
{
    int n,i;
    int at[10],bt[10],rt[10],q[10];
    int ct[10],tat[10],wt[10];
    int tq,current_time=0,completed=0;

    float avg_tat=0,avg_wt=0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Arrival Time:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&at[i]);
    }
}
```

```
printf("Enter Burst Time:\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    scanf("%d",&bt[i]);
```

```
    rt[i]=bt[i];
```

```
}
```

```
printf("Enter Queue Number :\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    scanf("%d",&q[i]);
```

```
}
```

```
printf("Enter Time Quantum: ");
```

```
scanf("%d",&tq);
```

```
while(completed < n)
```

```
{
```

```
    int executed = 0;
```

```
// FIRST: QUEUE 1 (ROUND ROBIN)
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    if(q[i]==1 && rt[i]>0 && at[i] <= current_time)
```

```
    {
```

```
        executed = 1;
```

```

    if(rt[i] > tq)
    {
        current_time += tq;
        rt[i] -= tq;
    }
    else
    {
        current_time += rt[i];
        rt[i] = 0;

        ct[i] = current_time;
        completed++;
    }
}

if(executed == 0)
{
    // SECOND: QUEUE 2 (FCFS)

    for(i=0;i<n;i++)
    {
        if(q[i]==2 && rt[i]>0 && at[i] <= current_time)
        {
            current_time += rt[i];
            rt[i] = 0;

```

```

        ct[i] = current_time;
        completed++;

        executed = 1;
        break;
    }
}

}

if(executed == 0)
    current_time++;
}

// calculating average TAT and WT
for(i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avg_tat += tat[i];
    avg_wt += wt[i];
}

printf("\nProcess\tQ\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{

```



```

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,q[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
    printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

    return 0;
}

```

OUTPUT:

```

NAME : R K MADAN          USN : 1WA24CS225          SECTION : '0'

Enter number of processes: 4
Enter Arrival Time:
0 0 0 10
Enter Burst Time:
4 3 8 5
Enter Queue Number :
1 1 2 1
Enter Time Quantum: 2

Process Q      AT      BT      CT      TAT      WT
P1      1      0      4      6      6      2
P2      1      0      3      7      7      4
P3      2      0      8      15     15      7
P4      1      10     5      20     10      5

Average Turnaround Time = 9.50
Average Waiting Time = 4.50

Process returned 0 (0x0)   execution time : 24.717 s
Press any key to continue.

```

OBSERVATION COPY:

17/03/2016

LAB PROGRAM 2

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories - system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:-

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i;
    int a[10], b[10], at[10], q[10], ct[10], tat[10], wt[10];
    int tq, current_time = 0, completed = 0;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter number of processes : ");
    scanf("%d", &n);
    printf("Enter Arrival Time :\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &at[i]);
    }
    printf("Enter Burst Time :\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        at[i] = bt[i];
    }
    printf("Enter Queue Number :\n");
```

```
for (i=0; i<n; i++)
{
    scanf("%d", &q[i]);
}
printf("Enter The Time Quantum: ");
scanf("%d", &tq);

while (completed < n)
{
    int executed = 0;
    // Queue 1 (Round Robin)
    for (i=0; i<n; i++)
    {
        if (q[i] == 1 && nt[i] > 0 && at[i] <= current_time)
        {
            executed = 1;
            if (nt[i] > tq)
            {
                current_time += tq;
                nt[i] -= tq;
            }
            else
            {
                current_time += nt[i];
                nt[i] = 0;
                at[i] = current_time;
                completed++;
            }
        }
    }
    if (executed == 0)
    {
        // Queue 2 (FCFS)
        for (i=0; i<n; i++)
        {
```

```

if (q[i] == 2 && nt[i] > 0 && at[i] <= current_time)
{
    current_time += nt[i];
    nt[i] = 0;
    ct[i] = current_time;
    completed++;
    executed = 1;
    break;
}
}
}
if (executed == 0)
{
    current_time++;
}
}

// calculating Average TAT & WT
for (i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    avg_tat += tat[i];
    avg_wt += wt[i];
}

printf("\n Process\t P\t AT\t BT\t CT\t TAT\t WT\n");
for (i = 0; i < n; i++)
{
    printf("P %d\t %d\t %d\t %d\t %d\t %d\t %d\n",
        i+1, q[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\n Average Turnaround Times = %.2f", avg_tat/n);
printf("\n Average Waiting Time = %.2f\n", avg_wt/n);
return 0;
}

```

output: Enter number of processes: 4

Enter Arrival Time:

0 0 0 10

Enter Burst Time:

4 3 8 5

Enter Queue Number:

1 1 2 1

Enter Time Quantum: 2

Process	Q	AT	BT	CT	TAT	WT
P ₁	1	0	4	6	6	2
P ₂	1	0	3	7	7	4
P ₃	2	0	8	15	15	7
P ₄	1	10	5	20	10	5

Average Turnaround Time = 9.50

Average Waiting Time = 4.50

LAB PROGRAM – 3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

a) Rate- Monotonic

CODE:

```
#include <stdio.h>
#include <math.h>

//TO CALCULATE GCD
int gcd(int a, int b)
{
    return (b == 0) ? a : gcd(b, a % b);
}

//TO CALCULATE LCM
int lcm(int a, int b)
{
    return (a * b) / gcd(a, b);
}

int main()
{
    int n, i;
    int bt[10], pt[10], rt[10];
    int hyper, time = 0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst Times:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
```

```

        rt[i] = 0; // initially no job released
    }

    printf("Enter Periods:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &pt[i]);
    }

// Calculate hyperperiod = LCM of all periods
hyper = pt[0];
for (i = 1; i < n; i++)
{
    hyper = lcm(hyper, pt[i]);
}

printf("LCM=%d\n\n", hyper);

printf("Rate Monotone Scheduling:\n");
printf("PID  Burst  Period\n");
for (i = 0; i < n; i++)
{
    printf("%d    %d    %d\n", i+1, bt[i], pt[i]);
}

// Utilization check
double U = 0;
for (i = 0; i < n; i++)
{
    U += (double)bt[i] / pt[i];
}
double bound = n * (pow(2.0, 1.0/n) - 1);

printf("\n%.6f <= %.6f => %s\n", U, bound, (U <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hyper);

// Timeline simulation (block-style)
int current_running = -1;
while (time < hyper)
{
// Release jobs
    for (i = 0; i < n; i++) {
        if (time % pt[i] == 0) {
            rt[i] = bt[i];
        }
    }
}

```

```

    }

// Pick highest priority (smallest period)
int min_period = 9999, index = -1;
for (i = 0; i < n; i++) {
    if (rt[i] > 0 && pt[i] < min_period) {
        min_period = pt[i];
        index = i;
    }
}

// Print only when event changes
if (index != current_running)
{
    if (index != -1)
    {
        printf("%dms onwards: Process %d running\n", time, index+1);
    }
else
    {
        printf("%dms onwards: CPU is idle\n", time);
    }
    current_running = index;
}

// Execute one unit if a process is running
if (index != -1)
{
    rt[index]--;
}

time++;
}

return 0;
}

```


OUTPUT:

```
"C:\Users\Admin\Desktop\1W  X + v
NAME : R K MADAN      USN : 1WA24CS225      SECTION : 'O'
Enter number of processes: 2
Enter Burst Times:
20 35
Enter Periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID  Burst  Period
1    20    50
2    35    100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)  execution time : 14.917 s
Press any key to continue.
|
```

OBSERVATION COPY:

27/3/2026

camlin

27

LAB PROGRAM 3

3. A write a C Program to simulate Real-Time CPU scheduling algorithms

a) Rate-Monotonic

```
CODE:- #include <stdio.h>
#include <math.h>

// gcd calculation
int gcd (int a, int b)
{
    return (b == 0) ? a : gcd (b, a % b);
}

// lcm calculations
int lcm (int a, int b)
{
    return (a * b) / gcd (a, b);
}

int main()
{
    int n, i;
    int bt[10], pt[10], rt[10];
    int hyper, time = 0;
    printf ("Enter number of processes : ");
    scanf ("%d", &n);
    printf ("Enter Burst Times : \n");
    for (i = 0; i < n; i++)
    {
        scanf ("%d", &bt[i]);
        rt[i] = 0;
    }
}
```

Date: _____

```

printf("Enter Periods : \n");
for (i=0; i<n; i++)
{
    scanf("%d", &pt[i]);

    hyper = pt[0];
    for (i=1; i<n; i++)
    {
        hyper = lcm(hyper, pt[i]);
    }
    printf("LCM = %d \n \n", hyper);
    printf("Rate Monotone Scheduling: \n");
    printf("P2D Burst Period \n");
    for (i=0; i<n; i++)
    {
        printf("%d | %d | %d \n", i+1, bt[i], pt[i]);
    }

    double U=0;
    for (i=0; i<n; i++)
    {
        U += (double) bt[i] / pt[i];
    }

    double bound = n * (pow(2.0, 1.0/n) - 1);
    printf("Is %f <= %f? %s \n", U, bound, (U <= bound) ?
        "true" : "false");
    printf("Scheduling occurs for %d ms \n \n", hyper);

    int worstRunning = -1;
    while (time < hyper)
    {
        for (i=0; i<n; i++)
        {
            if (time % pt[i] == 0)

```

```
{
    pt[i] = bt[i];
}
}

int min_period = 9999, index = -1;
for (i = 0; i < n; i++)
{
    if (pt[i] > 0 && pt[i] < min_period)
    {
        min_period = pt[i];
        index = i;
    }
}

if (index != current_running)
{
    if (index != -1)
    {
        printf("%d ms onwards : Process %d running\n", time, index+1);
    }
    else
    {
        printf("%d ms onwards : CPU is idle\n", time);
    }
    current_running = index;
}

if (index != -1)
{
    pt[index] = -1;
}

time++;
}

return 0;
}
```

Output: Enter number of processes: 2

Enter Burst Times:

20 35

Enter Periods:

50 100

LCM = 100

Rate Monotone Scheduling:

PID	Burst	Period
-----	-------	--------

1	20	50
---	----	----

2	35	100
---	----	-----

$0.750000 \leq 0.828427 \Rightarrow \text{True}$
Scheduling occurs for 100ms

0ms onwards: 1 running

20ms onwards: 2 running

50ms onwards: 1 running

70ms onwards: 1 running

75ms onwards: CPU is idle

b) Earliest-deadline First

CODE:

```
#include <stdio.h>

int main()
{
    int n, i;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    int bt[n], dl[n], p[n], rem[n];
    printf("Enter Burst times:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rem[i] = 0; // initially no job released
    }

    printf("Enter Deadlines:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &dl[i]);
    }

    printf("Enter Periods:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &p[i]);
    }
```

```
printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], dl[i], p[i]);
}
```

```
int total_time = p[0];
for (i = 1; i < n; i++) {
    if (p[i] > total_time)
        total_time = p[i];
}
```

```
printf("\nScheduling occurs for %d ms\n\n", total_time);
```

```
int prev_idle = 0;
```

```
for (int time = 0; time < total_time; time++) {
    for (i = 0; i < n; i++) {
        if (time % p[i] == 0) {
            rem[i] = bt[i];
        }
    }
}
```

```
// Select task with earliest deadline
```

```
int selected = -1;
int min_deadline = 9999;
for (i = 0; i < n; i++) {
    if (rem[i] > 0) {
        if (dl[i] < min_deadline) {
            min_deadline = dl[i];
```

```

        selected = i;
    }
}

// Print timeline
if (selected == -1) {
    if (!prev_idle) {
        printf("%dms onwards: CPU is idle.\n", time);
        prev_idle = 1;
    }
} else {
    printf("%dms onwards: Task %d is running.\n", time, selected + 1);
    rem[selected]--;
    prev_idle = 0;
}

return 0;
}

```


OUTPUT:

```
"C:\Users\Admin\Desktop\1W X + v

PID    Burst  Deadline  Period
1       3      7         20
2       2      4         5
3       2      8        10

Scheduling occurs for 20 ms

0ms onwards: Task 2 is running.
1ms onwards: Task 2 is running.
2ms onwards: Task 1 is running.
3ms onwards: Task 1 is running.
4ms onwards: Task 1 is running.
5ms onwards: Task 2 is running.
6ms onwards: Task 2 is running.
7ms onwards: Task 3 is running.
8ms onwards: Task 3 is running.
9ms onwards: CPU is idle.
10ms onwards: Task 2 is running.
11ms onwards: Task 2 is running.
12ms onwards: Task 3 is running.
13ms onwards: Task 3 is running.
14ms onwards: CPU is idle.
15ms onwards: Task 2 is running.
16ms onwards: Task 2 is running.
17ms onwards: CPU is idle.

Process returned 0 (0x0)   execution time : 33.521 s
Press any key to continue.
```

OBSERVATION COPY:

camlin

31

02/3/2024

3.B Earliest Deadline First

CODE: #include <stdio.h>
int main()
{
int n, i;
printf("Enter number of tasks :");
scanf("%d", &n);
int bt[n], dl[n], p[n], rem[n];
printf("Enter Burst times : \n");
for (i = 0; i < n; i++)
{
scanf("%d", &bt[i]);
rem[i] = 0;
}
printf("Enter Deadlines : \n");
for (i = 0; i < n; i++)
{
scanf("%d", &dl[i]);
}
printf("Enter Periods : \n");
for (i = 0; i < n; i++)
{
scanf("%d", &p[i]);
}
printf("\n P\tD\tBurst\tDeadline\tPeriod\n");
for (i = 0; i < n; i++)
{
printf("%d\t%d\t%d\t%d\t%d\n", i+1, bt[i], dl[i], p[i]);
}
int total_time = p[0];
for (i = 1; i < n; i++)
{
if (p[i] > total_time)
total_time = p[i];
}

```

3
printf("In Scheduling occurs for %d ms\n\n", total_time);
int prev_idle = 0;
for (int time = 0; time < total_time; time++)
{
    for (i = 0; i < n; i++) {
        if (time - p[i] == 0) {
            mem[i] = b[i];
        }
    }
}

int selected = -1;
int min_deadline = 9999;
for (i = 0; i < n; i++) {
    if (mem[i] > 0) {
        if (d[i] < min_deadline) {
            min_deadline = d[i];
            selected = i;
        }
    }
}

if (selected == -1) {
    if (!prev_idle) {
        printf("%d ms onwards : CPU is idle.\n", time);
        prev_idle = 1;
    }
} else {
    printf("%d ms onwards : Task %d is running.\n", time, selected + 1);
    mem[selected]--;
    prev_idle = 0;
}
}
return 0;
}

```

INPUT: Enter number of tasks: 3

Enter Burst times:

3 2 2

Enter Deadlines:

7 4 8

Enter Periods:

20 5 10

PID	Burst	Deadline	Period
1	3	7	20
2	2	4	5
3	2	8	10

Scheduling occurs for 20ms

0ms onwards: Task 2 is running

1ms onwards: Task 2 is running

2ms onwards: Task 1 is running

3ms onwards: Task 1 is running

4ms onwards: Task 1 is running

5ms onwards: Task 2 is running

6ms onwards: Task 2 is running

7ms onwards: Task 3 is running

8ms onwards: Task 3 is running

9ms onwards: CPU is idle

10ms onwards: Task 2 is running

11ms onwards: Task 2 is running

12ms onwards: Task 3 is running

13ms onwards: Task 3 is running

14ms onwards: CPU is idle

15ms onwards: Task 2 is running

16ms onwards: Task 2 is running

17ms onwards: CPU is idle.

c) Proportional scheduling

CODE:

```
// Proportional Share Scheduling (Lottery Scheduling)

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Define Process structure
typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");
    scanf("%d", &n);

    Process p[n];
    srand(time(NULL)); // Seed random number generator

    // Input process details
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
```

```

printf("Tickets: ");
scanf("%d", &p[i].tickets);

total_tickets += p[i].tickets;
total_T += p[i].tickets;
}

printf("\n--- Proportional Share Scheduling ---\n");

// Input scheduling time period
int m;
printf("Enter the Time Period for scheduling: ");
scanf("%d", &m);

// Lottery scheduling simulation
for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index = -1;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n",
        winning_ticket, p[winner_index].name);
}

```

```

    }

    // Print proportional share of CPU time
    for (int i = 0; i < n; i++) {
        printf("\nThe Process: %s gets %.2f%% of Processor Time.\n",
            p[i].name, ((p[i].tickets / total_T) * 100));
    }

    return 0;
}

```

OUTPUT:

```

C:\Users\Admin\Desktop\1W X + v
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 56, Winner: P3
Tickets picked: 10, Winner: P1
Tickets picked: 5, Winner: P1
Tickets picked: 34, Winner: P3
Tickets picked: 38, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 13.316 s
Press any key to continue.

```


OBSERVATION COPY:

10/4/2016

camlin
Date: _____

c) Proportional scheduling:-

code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct {
    char name[5];
    int tickets;
} process;

int main()
{
    int n, total_tickets = 0;
    float total_T = 0.0;
    printf("Enter the number of Processes :");
    scanf("%d", &n);
    process p[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
    {
        printf("\n Process %d:\n", i+1);
        sprintf(p[i].name, "P%d", i+1);
        printf("Tickets:");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
    printf("\n --- Proportional share Scheduling ---\n");
    int m;
    printf("Enter the Time Period for Scheduling :");
    scanf("%d", &m);
    for (int i = 0; i < m; i++) {
        int winning_ticket = rand() % total_tickets + 1;
        int accumulated_ticket = 0;
        int winner_index = -1;
```



```

for (int j = 0; j < n; j++) {
    accumulated_tickets += p[j].tickets;
    if (winning_ticket <= accumulated_tickets) {
        winner_index = j;
        break;
    }
}

printf("Tickets picked: %d, winner: %s\n",
    winning_ticket, p[winner_index].name);

for (int i = 0; i < n; i++) {
    printf("In the Process %s gets %.2f % of Procession
        time\n", p[i].name, ((p[i].tickets / total_T) *
        100));
}

return 0;
}

```

Output Enter the number of Processes: 3

Process: 1

Tickets: 10

Process: 2

Tickets: 20

Process: 3

Tickets: 30

--- Proportional share Scheduling ---

Given the time Period for scheduling: 5

Tickets picked: 56, winner: P3

Tickets picked: 10, winner: P1

Tickets picked: 5, winner: P1

Tickets picked: 34, winner: P3

Tickets picked: 38, winner: P3

The Process: P1 gets 16.67% of procession Time

The process: P2 gets 33.33% of procession Time

The process: P3 gets 50.00% of procession Time

LAB PROGRAM – 4

Question:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores.

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty; // counts empty slots
sem_t full; // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

// Producer function
void* producer(void* arg) {
    int item, i = 0;
    while (1) {
        item = i++; // produce an item (here just incrementing number)
```

```

sem_wait(&empty);          // wait for empty slot
pthread_mutex_lock(&mutex); // enter critical section

buffer[in] = item;
printf("Produced: %d at buffer[%d]\n", item, in);
in = (in + 1) % BUFFER_SIZE;

pthread_mutex_unlock(&mutex); // leave critical section
sem_post(&full);              // signal that new item is available

sleep(1); // simulate time to produce
}
return NULL;
}

// Consumer function
void* consumer(void* arg) {
    int item;
    while (1) {
        sem_wait(&full);          // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex); // leave critical section
        sem_post(&empty);           // signal that a slot is free

        sleep(2); // simulate time to consume
    }
}

```

```

    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    // Initialize semaphores and mutex
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    // Create producer and consumer threads
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Wait for threads to finish (they won't in this infinite loop)
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Cleanup (not reached in this infinite loop)
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

OUTPUT:

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
Consumed: 14 from buffer[4]
Produced: 19 at buffer[4]
Consumed: 15 from buffer[0]
Produced: 20 at buffer[0]
Consumed: 16 from buffer[1]
Produced: 21 at buffer[1]
Consumed: 17 from buffer[2]
Produced: 22 at buffer[2]
Consumed: 18 from buffer[3]
Produced: 23 at buffer[3]
Consumed: 19 from buffer[4]
Produced: 24 at buffer[4]
Consumed: 20 from buffer[0]
Produced: 25 at buffer[0]
Consumed: 21 from buffer[1]
Produced: 26 at buffer[1]
Consumed: 22 from buffer[2]
Produced: 27 at buffer[2]
```

OBSERVATION COPY:

4 A. Bounded Buffer Problem (Producer Consumer Problem).

```
CODE: #include <stdio.h>
#include <stdlib.h>
int stack[100], top = -1;
int queue[100], front = -1, rear = -1, max;
int itemcounter = 0;

void produce()
{
    if (rear == max-1)
    {
        printf("Buffer is full! Cannot produce more items.\n");
        return;
    }
    if (front == -1)
        front = 0;
    itemcounter++;
    queue[++rear] = itemcounter;
    printf("Producer has produced : item %d\n", itemcounter);
}

void consume()
{
    if (front == -1 || rear == -1)
    {
        printf("Buffer is empty.\n");
        return;
    }
    printf("Consumer has consumed : item %d\n", queue[front]);
    if (rear < front)
        front = rear = -1;
}
```

```
int main()
{
    int choice;
    printf("Enter the size of the buffer : ");
    scanf("%d", &max);
    if(max > 100)
    {
        printf("Max size is 100.\n");
        return 0;
    }
    while(1)
    {
        printf("\n Enter 1. Produce 2. Consume 3. Exit\n");
        printf("Enter your choice : ");
        scanf("%d", &choice);
        switch(choice)
        {
            case 1:
                produce();
                break;
            case 2:
                consume();
                break;
            case 3:
                printf("Exiting Program...\n");
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```


Output:

Enter the size of the Buffer : 2

Enter

1. Producer

2. Consumer

3. Exit

Enter your choice : 1

producer has produced : item 1

Enter

"

Enter your choice : 1

producer has produced : item 2

Enter

"

Enter your choice : 1

Buffer is full ! Cannot produce more item

Enter

"

Enter your choice : 2

consumer has consumed : item 2

Enter

2. Producer

2. Consumer

3. Exit

Enter your choice : 3

Exiting Program...

b) Dining-Philosopher's problem

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N];    // One mutex per fork
pthread_t philosophers[N];   // Philosopher threads

// Philosopher routine
void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;           // left fork index
    int right = (id + 1) % N; // right fork index

    while (1) {
        // Thinking
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        // To avoid deadlock:
        // Even philosophers pick left then right
        // Odd philosophers pick right then left
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
```

```

        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d.\n", id, right);
    } else {
        pthread_mutex_lock(&forks[right]);
        printf("Philosopher %d picked up right fork %d.\n", id, right);

        pthread_mutex_lock(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);
    }

    // Eating
    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    // Put down forks
    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);
    printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}

return NULL;
}

int main() {
    int i;
    int ids[N];

    // Initialize forks (mutexes)
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }
}

```

```
    ids[i] = i;
}

// Create philosopher threads
for (i = 0; i < N; i++) {
    pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
}

// Join threads (infinite loop, but good practice)
for (i = 0; i < N; i++) {
    pthread_join(philosophers[i], NULL);
}

// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++) {
    pthread_mutex_destroy(&forks[i]);
}

return 0;
}
```

OUTPUT:

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 is thinking.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
```

OBSERVATION COPY:

4B Dining Philosophy

CODES

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_t forks[N];
pthread_t philosophers[N];
void * philosopher(void * num)
{
    int id = *(int *) num;
    int left = id;
    int right = (id + 1) % N;
    while (1)
    {
        printf ("Philosopher %d is thinking\n", id);
        sleep(1);
        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);
            printf ("Philosopher %d picked up left fork %d\n",
                id, left);
            pthread_mutex_lock(&forks[right]);
            printf ("Philosopher %d picked up right fork %d\n",
                id, right);
        }
        else
        {
            pthread_mutex_lock(&forks[right]);
            printf ("Philosopher %d picked up right fork %d\n",
                id, right);
            pthread_mutex_lock(&forks[left]);
            printf ("Philosopher %d picked up left fork %d\n",
                id, left);
        }
    }
}

```

```

printf("Philosopher %d is eating\n", id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}
return NULL;
}

```

```

int main()
{
    int i;
    int ids[N];
    for (i=0; i<N; i++)
    {
        pthread_mutex = init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i=0; i<N; i++)
    {
        pthread = create(&philosophers[i], NULL, philosopher,
                        &ids[i]);
    }
    for (i=0; i<N; i++)
    {
        pthread = join(philosophers[i], NULL);
    }
    for (i=0; i<N; i++)
    {
        pthread_mutex = destroy(&forks[i]);
    }
    return 0;
}

```

✓
copy

OUTPUT:

philosopher 0 is thinking
 philosopher 1 is thinking
 philosopher 2 is thinking
 philosopher 3 is thinking
 philosopher 4 is thinking

⋮
 ⋮
 ⋮



LAB PROGRAM – 5

Question:

Write a C program to simulate:

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance.

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m];
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        for(j = 0; j < m; j++)
```

```
        {
```



```

        scanf("%d", &alloc[i][j]);
    }
}

printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}

for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n], work[m];

for(i = 0; i < n; i++)

```

```

{
    finish[i] = 0;
}

for(i = 0; i < m; i++)
{
    work[i] = avail[i];
}

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }
        }
    }
}

```

```

        if(possible)
        {
            for(k = 0; k < m; k++)
            {
                work[k] += alloc[i][k];
            }

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0)
{
    break;
}

if(count == n)
{
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);

        if(i != n - 1)

```

```

        {
            printf(" -> ");
        }
    }
}
else
{
    printf("\nSystem is NOT in a safe state.");
}

return 0;
}

```

OUTPUT:

```

NAME : R K MADAN          USN : 1WA24CS225          SECTION : '0'

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)   execution time : 52.958 s
Press any key to continue.
|

```

OBSERVATION COPY:

camlin
Date: _____

49

15/12/2026

LAB PROGRAM: 5

SA.] Write a C program to simulate Banker's Algorithm for the purpose of deadlock avoidance (Safety Algorithm)

→ Safety Algorithm:

1. Let $work$ & $finish$ be vectors of length m & n
 initialize $work = Available$
 $Finish[i] = false$ for $i = 0$ to $n-1$
2. Find an index i such that both:
 - a) $Finish[i] = false$
 - b) $Need \leq work$
3. If no such i exists, go to step 4
 $work = work + Allocation_i$
 $Finish[i] = true$
 go to step 2
4. If $finish[i] == true$ for all i , then the system is in a safe state.

→ Example:

Process	Allocation			Max			available work			(Max-Allocation) Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	0	1	0	7	5	3	3	3	2	7	4	3
P ₁	2	0	0	3	2	2				1	2	2
P ₂	3	0	2	9	0	2				6	0	0
P ₃	2	1	1	2	2	2				0	1	1
P ₄	0	0	2	4	3	3				4	3	1

Trace :-

1. $work = available = 3\ 3\ 2$
 $finish[i] = false$ for $i = 0$ to $n(4)$
2. $finish[0] = false$ $P_0 \rightarrow$ Not Safe
 $Need_0 \leq work$
 $\Rightarrow 7\ 4\ 3 \leq 3\ 3\ 2$ (False)
 $finish[0] = true$
2. $finish[1] = false$ $P_1 \rightarrow$ safe
 $Need_1 \leq work$
 $\Rightarrow 1\ 2\ 2 \leq 3\ 3\ 2$ (True)
 $\therefore work = 3\ 3\ 2 + 2\ 0\ 0 \Rightarrow 5\ 3\ 2 //$
 $finish[1] = true$
2. $finish[2] = false$ $P_2 \rightarrow$ Not safe
 $Need_2 \leq work$
 $6\ 0\ 0 \leq 5\ 3\ 2$ (False)
 $finish[2] = true$
2. $finish[3] = false$ $P_3 \rightarrow$ safe
 $Need_3 \leq work$
 $0\ 1\ 1 \leq 5\ 3\ 2$ (True)
 $\therefore work = 5\ 3\ 2 + 2\ 1\ 1 \Rightarrow 7\ 4\ 3$
 $finish[3] = true$
2. $finish[4] = false$ $P_4 \rightarrow$ safe
 $Need_4 \leq work$
 $1\ 4\ 3\ 1 \leq 7\ 4\ 3$ (True)
 $\therefore work = 7\ 4\ 3 + 0\ 0\ 2 \Rightarrow 7\ 4\ 5 //$
2. $finish[0] = false$ $P_0 \rightarrow$ safe
 $Need_0 \leq work$
 $7\ 4\ 3 \leq 7\ 4\ 5$ (True)

$$\therefore \text{work} = 745 + 010 \Rightarrow 755 //$$

2. finish[2] = false

$$\text{Need}_2 \leq \text{work}$$

$$600 \leq 755 \text{ (true)}$$

$$\therefore \text{work} = 755 + 302 \Rightarrow 1057$$

finish[2] = true.

$\therefore$ the safe sequence is $P_1, P_3, P_4, P_0, P_2 //$

PROGRAM

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, m, i, j, k;
```

```
printf("Enter number of processes :");
```

```
scanf("%d", &n);
```

```
printf("Enter number of resources :");
```

```
scanf("%d", &m);
```

```
int alloc[n][m], max[n][m], need[n][m], avail[m];
```

```
printf("Enter Allocation Matrix : \n");
```

```
for (i=0; i<n; i++) {
```

```
for (j=0; j<m; j++) {
```

```
scanf("%d", &alloc[i][j]);
```

```
}
```

```
printf("Enter Maximum Demand Matrix : \n");
```

```
for (i=0; i<n; i++) {
```

```
for (j=0; j<m; j++) {
```

```
scanf("%d", &max[i][j]);
```

```
}
```

```
printf("Enter Available Resources : \n");
```

```
for (i=0; i<m; i++)
```

```
{ scanf("%d", &avail[i]);
```

```
}
```


camlin
Date: _____

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = max[i][j] - avail[i][j];
    }
}

int finish[n], safeSeq[n], work[m];
for (i=0; i<n; i++) {
    finish[i] = 0;
}

for (i=0; i<m; i++) {
    work[i] = available[i];
}

int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int possible = 1;
            for (j=0; j<m; j++) {
                if (need[i][j] > work[j]) {
                    possible = 0;
                    break;
                }
            }
            if (possible) {
                for (k=0; k<m; k++) {

```



```
work[K] += alloc[i][K];  
}  
safeSeq[count++] = i;  
finish[i] = 1;  
found = 1;  
}  
}  
}  
if (found == 0)  
{  
    break; }  
}  
if (count == n)  
{  
    printf("System is in a safe state.\n");  
    printf("Safe sequence is: ");  
    for (i = 0; i < n; i++)  
    {  
        printf(" P-1.d", safeSeq[i]);  
        if (i != n-1)  
        {  
            printf("→");  
        }  
    }  
}  
else {  
    printf("\n System is NOT in a safe state.");  
}  
return 0;  
}
```

Output: Enter number of processes : 5
Enter number of resources : 3
Enter Allocation Matrix :
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix :
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources :
3 3 2
System is in a safe State.
Safe Sequence is : $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

b) Deadlock Detection

CODE:

```
#include <stdio.h>

int main()
{
    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    int n, m, i, j, k;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m];
    int avail[m];

    printf("Enter the allocation matrix:\n");

    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);

        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }
}
```

```
printf("Enter the request matrix:\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("Process %d: ", i);
```

```
    for(j = 0; j < m; j++)
```

```
    {
```

```
        scanf("%d", &request[i][j]);
```

```
    }
```

```
}
```

```
printf("Enter the available resources: ");
```

```
for(i = 0; i < m; i++)
```

```
{
```

```
    scanf("%d", &avail[i]);
```

```
}
```

```
int finish[n], safeSeq[n], work[m];
```

```
for(i = 0; i < m; i++)
```

```
{
```

```
    work[i] = avail[i];
```

```
}
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    finish[i] = 0;
```

```

}

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {
                for(k = 0; k < m; k++)
                {
                    work[k] += alloc[i][k];
                }
            }
        }
    }
}

```

```

        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}
}

if(found == 0)
{
    break;
}
}

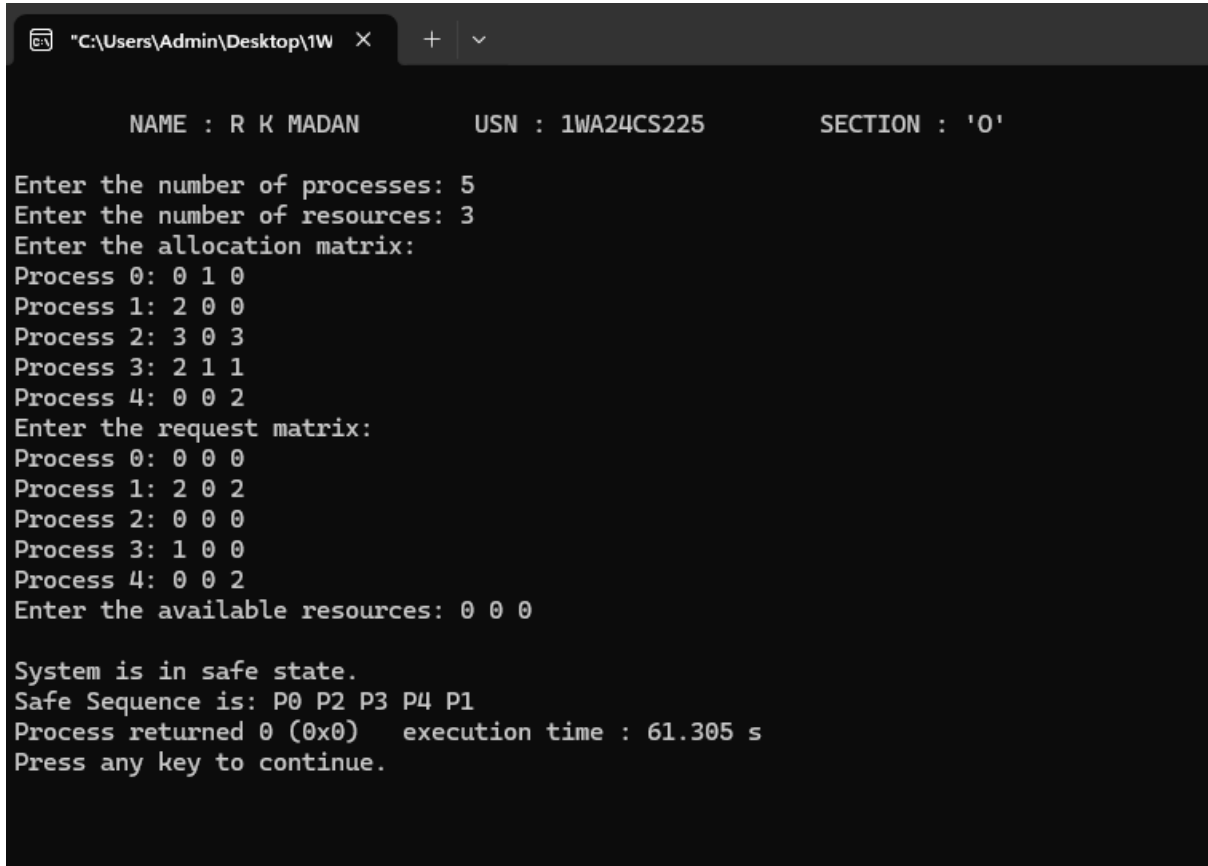
if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d ", safeSeq[i]);
    }
}
else
{
    printf("\nDeadlock detected.\n");
}

return 0;
}

```

OUTPUT:



```
"C:\Users\Admin\Desktop\1W" X + v

NAME : R K MADAN      USN : 1WA24CS225      SECTION : '0'

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
Process returned 0 (0x0)   execution time : 61.305 s
Press any key to continue.
```

OBSERVATION COPY:

camlin
Date: _____

55

58 Write a C program to simulate Deadlock Detection.

→ Deadlock Detection Algorithm: (Banker's Algorithm)

- Let $work$ & $finish$ be vectors of length m & n
 - $work = Available$
 - for $i = 0$ to n ,
 - if $Allocation \neq 0$, then
 $Finish[i] = false$, otherwise
 $Finish[i] = true$
- Find an index i such that
 - $Finish[i] == false$
 - $Request[i] \leq work$
 If no such i exists go to step 4.
- $work = work + Allocation[i]$
 $Finish[i] = true$
 go to step 2.
- if $Finish[i] == false$, for some i , $1 \leq i \leq n$,
 The system is in deadlock state.
 Moreover, if $Finish[i] == false$,
 Then P_i is deadlocked.

→ Example:

Process	Allocation			Request			work		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

→ Tracing :-

1 a) $work = available = 000$

b) for $i = 0$ to $n-1$ (4)

$finish[i] = false$

2. $i = 0$

$P_0 \rightarrow safe$

$finish[0] = false$

$Request_0 \leq work$

$\Rightarrow 000 \leq 000$ (true)

$\therefore work = 000 + 010 \Rightarrow 010 //$

$finish[0] = true$

2. $i = 1$

$P_1 \rightarrow Not\ safe$

$finish[1] = false$

$Request_1 \leq work$

$\Rightarrow 202 \leq 010$ (false)

$\therefore finish[1] = false$

2. $i = 2$

$P_2 \rightarrow safe$

$finish[2] = false$

$Request_2 \leq work$

$\Rightarrow 000 \leq 010$ (true)

$\therefore work = 010 + 303 \Rightarrow 313 //$

$\therefore finish[2] = true$

2. $i = 3$

$P_3 \rightarrow safe$

$finish[3] = false$

$Request_3 \leq work$

$100 \leq 313$ (true)

$\therefore work = 313 + 211 \Rightarrow 524 //$

$\therefore finish[3] = true$

2. $i = 4$

$P_4 \rightarrow safe$

$finish[4] = false$

$Request_4 \leq work$

$\Rightarrow 002 \leq 524$ (true)

$\therefore \text{work} = 524 + 00 \times 2 \Rightarrow 526 //$

$\therefore \text{finish}[4] = \text{true}$

$P_1 \rightarrow \text{safe}$

2. $i = 1$

$\text{finish}[i] \neq \text{false}$

$\text{Request}[i] \leq \text{work}$

$\Rightarrow 202 \leq 526$ (true)

$\therefore \text{work} = 526 + 200 \Rightarrow 726 //$

$\therefore \text{finish}[1] = \text{true}$

$\therefore$ for $i = 0$ to 4 , $\text{finish}[i] = \text{true}$, $\therefore$ there's no deadlock

$\therefore$ Safe sequence :- $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

Program Code:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, m, i, j, k;
```

```
printf("Enter the number of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter the number of resources: ");
```

```
scanf("%d", &m);
```

```
int alloc[n][m], request[n][m], avail[m];
```

```
printf("Enter the allocation matrix: \n");
```

```
for (i = 0; i < n; i++)
```

```
{ printf("Process %d: ", i);
```

```
for (j = 0; j < m; j++)
```

```
{
```

```
scanf("%d", &alloc[i][j]);
```

```
}
```

```
}
```

```

printf("Enter the request matrix : \n");
for (i=0; i<n; i++)
{ printf("Process %d : ", i);
  for (j=0; j<m; j++)
  {
    scanf("%d", &request[i][j]);
  }
}

printf("Enter the available resources : ");
for (i=0; i<m; i++)
{
  scanf("%d", &avail[i]);
}

int finish[n], safe_seq[n], work[m];
for (i=0; i<m; i++)
{
  work[i] = avail[i];
}

for (i=0; i<n; i++)
{
  finish[i] = 0;
}

int count = 0;
while (count < n)
{
  int found = 0;
  for (i=0; i<n; i++)
  {
    if (finish[i] == 0)
    {
      int possible = 1;
      for (j=0; j<m; j++)
      {
        if (request[i][j] > work[j])

```

```
break;
}
if (possible)
{
    for (k = 0; k < m; k++)
    {
        work[k] += alloc[i][k];
    }
    safeSeq[count++] = i;
    finish[i] = 1;
    found = 1;
}
}
if (found == 0)
{
    break;
}
if (count == n)
{
    printf("In System is in safe state .\n");
    printf("Safe sequence is : ");
    for (i = 0; i < n; i++)
    {
        printf("P %d ", safeSeq[i]);
    }
}
else
{
    printf("In Deadlock detected .\n");
}
return 0;
}
```


Output: Enter the number of processes : 5
Enter the number of resources : 3
Enter the allocation matrix :
Process 0 : 0 1 0
Process 1 : 2 0 0
Process 2 : 3 0 3
Process 3 : 2 1 1
Process 4 : 0 0 2
Enter the request matrix :
Process : 0 : 0 0 0
Process : 1 : 2 0 2
Process : 2 : 0 0 0
Process : 3 : 1 0 0
Process : 4 : 0 0 2
Enter the available resources : 0 0 0
System is in safe state.
Safe Sequence is : P₀ P₂ P₃ P₄ P₁

~~18~~
8/5/26

LAB PROGRAM – 6

Question :

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

CODE:

```
#include <stdio.h>

int main()
{
    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    int b[20], p[20];
    int temp[20];
    int i, j, n, m;
    int allocation[20];

    printf("Enter number of memory blocks: ");
    scanf("%d", &n);

    printf("Enter sizes of %d memory blocks:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &b[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &m);
```

```

printf("Enter sizes of %d processes:\n", m);
for(i = 0; i < m; i++)
{
    scanf("%d", &p[i]);
}

// FIRST FIT

for(i = 0; i < n; i++)
    temp[i] = b[i];

for(i = 0; i < m; i++)
    allocation[i] = -1;

for(i = 0; i < m; i++)
{
    for(j = 0; j < n; j++)
    {
        if(temp[j] >= p[i])
        {
            allocation[i] = j;
            temp[j] = -1;
            break;
        }
    }
}

printf("\n--- First Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

```

```

for(i = 0; i < m; i++)
{
    printf("%d\t\t%d\t\t", i + 1, p[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

// BEST FIT

for(i = 0; i < n; i++)
    temp[i] = b[i];

for(i = 0; i < m; i++)
    allocation[i] = -1;

for(i = 0; i < m; i++)
{
    int best = -1;

    for(j = 0; j < n; j++)
    {
        if(temp[j] >= p[i])
        {
            if(best == -1 || temp[j] < temp[best])
            {
                best = j;
            }
        }
    }
}

```



```

    }
}

if(best != -1)
{
    allocation[i] = best;
    temp[best] = -1;
}
}

printf("\n--- Best Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < m; i++)
{
    printf("%d\t%d\t", i + 1, p[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

// WORST FIT

for(i = 0; i < n; i++)
    temp[i] = b[i];

for(i = 0; i < m; i++)
    allocation[i] = -1;

```

```

for(i = 0; i < m; i++)
{
    int worst = -1;

    for(j = 0; j < n; j++)
    {
        if(temp[j] >= p[i])
        {
            if(worst == -1 || temp[j] > temp[worst])
            {
                worst = j;
            }
        }
    }

    if(worst != -1)
    {
        allocation[i] = worst;
        temp[worst] = -1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < m; i++)
{
    printf("%d\t%d\t", i + 1, p[i]);

```

```

        if(allocation[i] != -1)

            printf("%d\n", allocation[i] + 1);

        else

            printf("Not Allocated\n");

    }

    return 0;
}

```

OUTPUT:

```

NAME : R K MADAN          USN : 1WA24CS225          SECTION : '0'

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.    Process Size    Block No.
1              212             2
2              417             5
3              112             3
4              426            Not Allocated

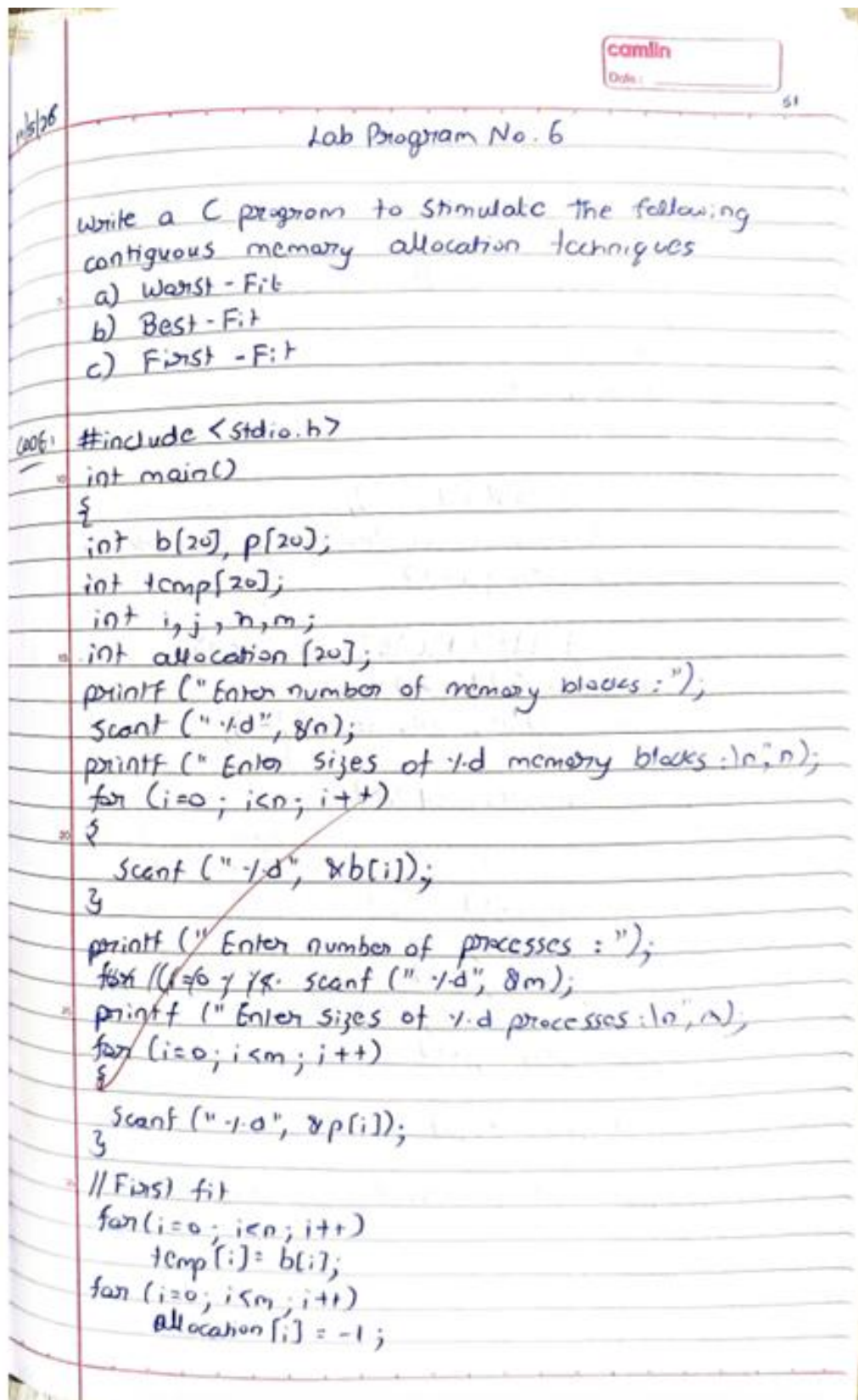
--- Best Fit ---
Process No.    Process Size    Block No.
1              212             4
2              417             2
3              112             3
4              426             5

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212             5
2              417             2
3              112             4
4              426            Not Allocated

Process returned 0 (0x0)   execution time : 51.544 s
Press any key to continue.
|

```

OBSERVATION COPY:



```

for (i=0; i<m; i++)
{
    for (j=0; j<n; j++)
    {
        if (temp[j] >= p[i])
        {
            allocation[i] = j;
            temp[j] = -1;
            break;
        }
    }
    printf("--- First Fit ---");
    printf("Process No. | Process Size | Block No. |");
    for (i=0; i<m; i++)
    {
        printf("/d |t|t /d |t|t", i+1, p[i]);
        if (allocation[i] != -1)
            printf("/d \n", allocation[i]+1);
        else
            printf("Not allocated \n");
    }
}

// Best Fit
for (i=0; i<n; i++)
    temp[i] = b[i];
for (j=0; j<m; j++)
    allocation[j] = -1;
for (j=0; j<n; j++)
{
    if (temp[j] >= p[j])
    {
        if (best == -1 || temp[j] < temp[best])
        {
            best = j;
        }
    }
}
}
}

```

```

if (best != -1)
{
    allocation[i] = best;
    temp[best] = -1;
}
}

printf("\n --- Best Fit ---\n");
printf("Process No. | Process Size | Block No. | \n");
for (i=0; i<m; i++)
{
    printf("%d | %d | %d |", i+1, p[i]);
    if (allocation[i] != -1)
        printf("%d |", allocation[i]+1);
    else
        printf("Not allocated | \n");
}

// worst Fit
for (i=0; i<m; i++)
    temp[i] = b[i];
for (i=0; i<m; i++)
    allocation[i] = -1;
for (i=0; i<m; i++)
{
    int worst = -1;
    for (j=0; j<n; j++)
    {
        if (temp[j] >= p[i])
        {
            if (worst == -1 || temp[j] > temp[worst])
            {
                worst = j;
            }
        }
    }
}
}
}

```



```

if (worst != -1)
{
    allocation[i] = worst;
    temp[worst] = -1;
}
}
printf("\n --- Worst Fit --- \n");
printf("Process No. | Process Size | Block No.\n");
for (i=0; i<m; i++)
{
    printf("%d | %d | %d\n", i+1, p[i], allocation[i]);
    if (allocation[i] != -1)
        printf("%d | %d | %d\n", i+1, p[i], allocation[i]+1);
    else
        printf("Not allocated\n");
}
return 0;
}

```

Output: Enter number of memory blocks: 5
 Enter sizes of 5 memory blocks:
 100 500 200 300 600
 Enter number of processes: 4
 Enter sizes of 4 processes:
 212 417 112 426
 --- First Fit ---

Process No.	Process size	Block No.
1	212	2
2	417	5
3	112	3
4	426	Not allocated.

--- Best Fit ---

Process No.	Process size	Block No.
1	212	4
2	417	2

3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	4
4	426	Not allocated

✓
212

LAB PROGRAM – 7

Question:

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

CODE:

```
#include <stdio.h>

int main()
{
    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
    int pages[50], frames[10];
    int n, f;
    int i, j, k;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    // ----- FIFO -----
```

```

int faults = 0;
int next = 0;
int found;

for(i = 0; i < f; i++)
{
    frames[i] = -1;
}

printf("\n--- FIFO Page Replacement ---\n");

for(i = 0; i < n; i++)
{
    found = 0;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }

    if(found == 0)
    {
        frames[next] = pages[i];
        next = (next + 1) % f;
        faults++;
    }
}

```

```

    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < f; j++)
    {
        if(frames[j] != -1)
        {
            printf("%d", frames[j]);

            if(j != f - 1)
            {
                printf(" ");
            }
        }
    }

    printf("]\n");
}

printf("Total Page Faults (FIFO): %d\n", faults);

// ----- LRU -----

int time[10];
int count = 0;
faults = 0;

for(i = 0; i < f; i++)
{

```

```

frames[i] = -1;
time[i] = 0;
}

printf("\n--- LRU Page Replacement ---\n");

for(i = 0; i < n; i++)
{
    found = 0;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            count++;
            time[j] = count;
            break;
        }
    }

    if(found == 0)
    {
        int pos = -1;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;

```

```

        break;
    }
}

if(pos == -1)
{
    pos = 0;

    for(j = 1; j < f; j++)
    {
        if(time[j] < time[pos])
        {
            pos = j;
        }
    }
}

frames[pos] = pages[i];
count++;
time[pos] = count;
faults++;
}

printf("Page %d -> [", pages[i]);

for(j = 0; j < f; j++)
{
    if(frames[j] != -1)
    {
        printf("%d", frames[j]);
    }
}

```

```

        if(j != f - 1)
        {
            printf(" ");
        }
    }
}

printf("]\n");
}

printf("Total Page Faults (LRU): %d\n", faults);

// ----- OPTIMAL -----

faults = 0;

for(i = 0; i < f; i++)
{
    frames[i] = -1;
}

printf("\n--- Optimal Page Replacement ---\n");

for(i = 0; i < n; i++)
{
    found = 0;

    for(j = 0; j < f; j++)
    {

```

```

    if(frames[j] == pages[i])
    {
        found = 1;
        break;
    }
}

if(found == 0)
{
    int pos = -1;

    for(j = 0; j < f; j++)
    {
        if(frames[j] == -1)
        {
            pos = j;
            break;
        }
    }

    if(pos == -1)
    {
        int farthest = -1;
        int index;

        for(j = 0; j < f; j++)
        {
            int found_future = 0;

            for(k = i + 1; k < n; k++)

```

```

        {
            if(frames[j] == pages[k])
            {
                if(k > farthest)
                {
                    farthest = k;
                    pos = j;
                }

                found_future = 1;
                break;
            }
        }

        if(found_future == 0)
        {
            pos = j;
            break;
        }
    }

    frames[pos] = pages[i];
    faults++;
}

printf("Page %d -> [", pages[i]);

for(j = 0; j < f; j++)
{

```



```
    if(frames[j] != -1)
    {
        printf("%d", frames[j]);

        if(j != f - 1)
        {
            printf(" ");
        }
    }

    printf("]\n");
}

printf("Total Page Faults (Optimal): %d\n", faults);

return 0;
}
```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W X + v

NAME : R K MADAN      USN : 1WA24CS225      SECTION : '0'

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [4 0 3]
Page 2 -> [4 0 2]
Page 3 -> [4 3 2]
Page 0 -> [0 3 2]
Page 3 -> [0 3 2]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7

Process returned 0 (0x0)   execution time : 57.823 s
Press any key to continue.
```

OBSERVATION COPY:

22/5/2026

LAB - PROGRAM No. 7

* Given references to the following Pages :-

5, 0, 1, 0, 2, 3, 0, 2, 4, 3, 3, 2, 0, 2, 1, 2, 7, 0, 1, 0

find Hit ratio & Miss ratio for

a) FIFO
b) optimal
c) LRU

USE Three frames

a) FIFO

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	0
f ₃			1	1	1	1	0	0	0	0	0	0	0	0	1	1	7	7	7	7
f ₂		0	0	0	0	3	3	3	3	3	3	2	2	2	2	2	2	0	0	0
f ₁	5	5	5	5	2	2	2	2	4	4	4	4	4	4	4	4	7	7	7	7
	*	*	*	Hit	*	*	*	Hit	*	Hit	Hit	*	Hit	Hit	*	Hit	*	*	Hit	Hit

Hit ratio $\Rightarrow \frac{\text{no. of hit}}{\text{no. of pages}} \Rightarrow \frac{9}{10} \Rightarrow$

Miss ratio $\Rightarrow \frac{\text{no. of miss}}{\text{no. of pages}} \Rightarrow \frac{11}{21} \Rightarrow$

b) optimal Page Replacement:

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	0
f ₃			1	1	1	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1
f ₂		0	0	0	0	0	0	0	4	4	4	4	0	0	0	0	0	0	0	0
f ₁	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7	7
	*	*	*	Hit	*	*	Hit	Hit	*	Hit	Hit	Hit	*	Hit	*	Hit	*	Hit	Hit	Hit

Hit ratio $\Rightarrow \frac{13}{21}$ & Miss ratio $\Rightarrow \frac{9}{21}$

c) LRU:

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	0
f ₃			1	1	1	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	
f ₂	0	0	0	0	0	0	0	0	4	4	4	4	0	0	0	0	0	0	0	0	
f ₁	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7	
	*	*	*	Hit	*	*	Hit	Hit	*	Hit	Hit	Hit	*	Hit	*	Hit	*	Miss	Hit	Hit	

Hit ratio $\Rightarrow$

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	0
f ₃			1	1	1	3	3	3	4	3	3	3	3	3	1	1	1	1	1	1	
f ₂	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	7	0	0	0	0	
f ₁	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	
	*	*	*	Hit	*	*	H	H	*	H	H	H	H	*	H	*	H	H	H	H	

	5	0	1	0	2	3	0	2	4	3	3	2	0	2	1	2	7	0	1	1	0
f ₃			1	1	1	3	3	3	4	4	4	4	0	0	0	0	7	7	7	7	
f ₂	0	0	0	0	0	0	0	0	3	3	3	3	3	1	1	1	0	0	0	0	
f ₁	5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	
	*	*	H	*	*	H	H	*	H	H	H	H	H	*	H	*	H	H	H	H	

$$\text{Hit ratio} \Rightarrow \frac{\text{no. of hit}}{\text{no. of pages}} \Rightarrow \frac{9}{21}$$

$$\text{Miss ratio} \Rightarrow \frac{\text{no. of Miss}}{\text{ratio}} \Rightarrow \frac{12}{21}$$

7A)B)C)

```
#include <stdio.h>
int main()
{
    int pages[50], frames[10];
    int n, f;
    int i, j, K;
    printf("Enter number of pages : ");
    scanf("%d", &n);
    printf("Enter page reference string: \n");
    for (i=0; i<n; i++)
    {
        scanf("%d", &pages[i]);
    }
    printf("Enter numbers of frames: ");
    scanf("%d", &f);

    // FIFO
    int faults = 0, next = 0, faults = 0;
    for (i=0; i<f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }
    printf("\n --- LRU Page Replacement --- \n");
    for (i=0; i<n; i++)
    {
        found = 0;
        for (j=0; j<f; j++)
        {
            if (frames[j] == pages[i])
            {
                found = 1;
                count++;
                time[j] = count;
                break;
            }
        }
    }
}
```



```
    }  
    if (found == 0)  
    {  
        int pos = -1;  
        for (j = 0; j < f; j++)  
        {  
            if (frames[j] == -1)  
            {  
                pos = j;  
                break;  
            }  
        }  
        if (pos == -1)  
        {  
            pos = 0;  
            for (j = 1; j < f; j++)  
            {  
                if (time[j] < time[pos])  
                {  
                    pos = j;  
                }  
            }  
        }  
        frames[pos] = pages[i];  
        count++;  
        time[pos] = count;  
        faults++;  
    }  
    printf("Page %d -> ", pages[i]);  
    for (j = 0; j < f; j++)  
    {  
        if (frames[j] != -1)  
        {  
            printf("/d", frames[j]);  
            if (j != f-1)  
            {  
                printf(" ");  
            }  
        }  
    }
```

```

24
printf("%d\n");
}
printf("Total Page Faults [LRU] : %d\n", faults);

// optimal
faults = 0;
for (i = 0; i < f; i++)
{
    frames[i] = -1;
}
printf("\n --- optimal Page Replacement ---\n");
for (i = 0; i < n; i++)
{
    found = 0;
    for (j = 0; j < f; j++)
    {
        if (frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if (found == 0)
    {
        int pos = -1;
        for (j = 0; j < f; j++)
        {
            if (frames[j] == -1)
            {
                pos = j;
                break;
            }
        }
    }
}

```

```
if (pos == -1)
{
    int farthest = -1;
    int index;
    for (j=0; j<f; j++)
    {
        int found-future = 0;
        for (K=i+1; K<n; K++)
        {
            if (frames[j] == pages[K])
            {
                if (K > farthest)
                {
                    farthest = K;
                    pos = j;
                }
                found-future = 1;
                break;
            }
        }
        if (found-future == 0)
        {
            pos = j;
            break;
        }
    }
    frames[pos] = pages[i];
    faults++;
}

printf("Page %d -> [" pages[i]);
for (j=0; j<f; j++)
{
    if (frames[j] != -1)
    {
        printf(" %d", frames[j]);
    }
}
```



```

if(j != f-1)
{
    printf(" ");
}
}
}
}
printf(" ");
}
printf("Total Page Faults (optimal) : %d\n", faults);
return 0;
}

```

Output: Enter number of pages : 12
 Enter page reference string :
 7 0 1 2 0 3 0 4 2 3 0 3
 Number of frames : 3

--- FIFO Page Replacement ---

page 7 → [7]
 page 0 → [7 0]
 page 1 → [7 0 1]
 page 2 → [2 0 1]
 page 0 → [2 0 1]
 page 3 → [2 3 1]
 page 0 → [2 3 0]
 page 4 → [4 3 0]
 page 2 → [4 2 0]
 page 3 → [4 2 3]
 page 0 → [0 2 3]
 page 3 → [0 2 3]

Page Faults : 10

--- LRU Page Replacement ---

page 7 → [7]
 page 0 → [7 0]
 page 1 → [7 0 1]
 page 2 → [2 0 1]
 page 0 → [2 0 1]

Page 3 $\rightarrow$ [2 0 3]

Page 0 $\rightarrow$ [2 0 3]

Page 4 $\rightarrow$ [4 0 3]

Page 2 $\rightarrow$ [4 0 3]

Page 3 $\rightarrow$ [4 3 2]

Page 0 $\rightarrow$ [0 3 2]

Page 3 $\rightarrow$ [0 3 2]

Page faults : 9

--- optimal Page Replacement ---

Page 7 $\rightarrow$ [7]

Page 0 $\rightarrow$ [7 0]

Page 1 $\rightarrow$ [7 0 1]

Page 2 $\rightarrow$ [2 0 1]

Page 0 $\rightarrow$ [2 0 1]

Page 3 $\rightarrow$ [2 0 3]

Page 0 $\rightarrow$ [2 0 3]

Page 4 $\rightarrow$ [2 4 3]

Page 2 $\rightarrow$ [2 4 3]

Page 3 $\rightarrow$ [2 4 3]

Page 0 $\rightarrow$ [0 4 3]

Page 3 $\rightarrow$ [0 4 3]

Page Faults : 7

✓
2/1 ✓

INDEX PAGE OF OBSERVATION BOOK

NAME <u>R.K. MADAN</u>			SUBJECT <u>Operating System LAB</u>	
STD. <u>4th Sem</u> DIV. <u>O' Section</u>			ROLL NO. <u>1WA24CS225</u>	SCHOOL <u>BMSCE</u>
SL. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
01	01-02	Practice Program	27/3/25	10
02	02-05	Lab Program 1a, FCFS	6/3/25	10
03	06-08	Lab Program 1b, SJF (Non-Pre-emptive)	6/3/25	10
04	09-11	Lab Program 1c, SRTF (SJF Pre-emptive)	6/3/25	
05	12-15	Lab Program 1d, Priority (Non-Pre-emptive)	13/3/25	
06	16-18	Lab Program 1e, Priority (Pre-emptive)	13/3/25	10
07	19-21	Lab Program 1f, Round Robin	13/3/25	
08	22-25	Lab Program 2, Multilevel Queue Scheduling	13/3/25	10
09	26-30	Lab Program 3a, Rate Monotonic Scheduling	27/3/25	10
10	31-33	Lab Program 3b, Earliest Deadline First	27/3/25	10
11	34-35	Lab Program 3c, Proportional Scheduling	10/4/25	10
12	36-42	Lab Program 4a, Producer-Consumer (Queue)	24/4/25	10
13	43-48	Lab Program 4b, Dining Philosophers (Threads)	24/4/25	10
14	49-54	Lab Program 5a, Safety Algorithm	8/5/25	10
15	55-60	Lab Program 5b, Deadlock Detection Algorithm	8/5/25	10
16	61-63	Lab Program 6, Longest Memory Allocation	14/5/25	10
17	64-73	Lab Program 7, Page Replacement Algorithm	22/5/25	10